



Enhancing Self-Driving Car Control Using CNN and Transfer Learning Models

Venkata Sai Reddy Peddireddy

Dr. CJ Wu, Ph.D.,

Advanced Machine Learning

Git link :

https://github.com/venkatasai8120/AML_Assignments.git

INDEX
Introduction
Problem Statement
Literature Review
Methodology
Dataset Description
Model Architectures
Training Process
Evaluation Metrics
Results
Model Comparison
Discussion
Improvements Made
Limitations
Future Work
Conclusion
References

INTRODUCTION

Deep learning has become one of the most powerful technologies within computer vision, as it allows computers to accurately analyze the contents of an image. Self-driving cars use some of the most popular and fastest-growing computer vision technology. Deep learning models are the primary components in self-driving cars used for perception, sensor fusion, and real-time decisions during navigation.

The first step of self-driving systems is predicting the steering angle of a car from images taken from the cameras placed in it. This problem is known as behavioral cloning because it is modeled as a supervised learning problem where a neural network learns to imitate a human being using images from road cameras. Newer deep learning architectures including convolutional neural networks (CNNs), residual networks (ResNet), and efficient architectures like MobileNet have made steering predictions much more accurate and stronger in adverse conditions.

The goal of this project was to apply state-of-the-art deep learning techniques to build, train and test various steering angle prediction models on the Udacity Self-Driving Car dataset. Seven different models were evaluated ranging from simple convolutional neural networks to complex transfer learning models to determine which model performs best at predicting steering angles. Through this project, we hope to better understand the advantages, disadvantages, and future potential of using deep learning for autonomous vehicle control.

PROBLEM STATEMENT

To drive safely, an autonomous vehicle must instantly choose a action for steering given the filtered video input. A steering prediction error can lead to lane drift, vehicle instability, or collisions. Customary rule-based algorithms do not account for various road scenes, shadows, curves, or changing environments at all.

What methods do you use deep learning models to predict steering angle using images from the front-facing camera? Which deep learning architecture performs the best for predicting steering angles?

- Many steering angles are around zero, leading to a class imbalance.
- Road images differ in brightness, shadow, and curvature.
- The model must also generalize to unseen validation images.
- Overfitting needs to be controlled for multiple architectures.

To address this problem, we trained seven deep learning models including NVIDIA end-to-end CNN, CNN with different configurations, ResNet18, MobileNetV2, and custom-made hybrid model. The purpose of this procedure was to compare the performance of the models by using MSE, RMSE, MAE and actual predictions in percentage.

The results provide an overview of the advantages and disadvantages of each architecture and which of the proposed architecture is best suited for predicting the steering of autonomous vehicles in real-time

LITERATURE REVIEW

Deep learning has advanced computer vision tasks, giving computers the ability to classify images more accurately. One of the first uses of CNNs in this sense was in image classification, which was later applied to autonomous driving. CNNs learn hierarchical spatial features such as edges and textures, and high-level representations, making them a common choice for road detection and steering prediction.

NVIDIA (Bojarski et al., 2016) first showed that a CNN could be trained on raw images of the road to learn to predict steering angles. The system showed that the learned model could drive without a large set of handcrafted driving rules, and it inspired behavior cloning methods. Since then, deeper networks like ResNet, MobileNet and Beginning networks have continued to show improvements in gradient flow, extracted features, and computation.

Transfer learning is widely used in driving. Using pretrained ResNet18 and MobileNetV2 models on millions of images with labels, rich features can be extracted, and models can be quickly adapted to driving datasets. These types of models are useful when data is scarce, or there are fewer training samples for certain conditions, as is the case with self-driving datasets, where most roads are straight.

In more recent work beyond CNN, Vision Transformers (ViTs) or hybrid ViT-CNN architectures have been used. These are seen as having greater potential to capture long-range dependencies valuable for capturing road geometry. However, these new approaches require much larger datasets and computational power.

Other prominent advancements include:

- Data augmentation to improve generalization for various lighting and weather conditions.
- Models with multiple views combine inputs from the center camera, the left camera, and the right camera.
- Sensors fuse images, LiDAR, radar, or vehicle telemetry through use.
- Attention mechanisms zoom in on road features like lane markings and curves.

Despite these advances, issues such as data overfitting, angle imbalance, robustness and interpretability on distribution escape remain. Researchers continue to optimize network architecture, generate synthetic data, and model for uncertainty to improve robustness in real-world autonomous driving scenarios.

Methodology

The general methodology to design, train and evaluate the deep learning model for steering angle prediction follows the steps of dataset preparation, model design, model training and quantitative model evaluation.

1. Dataset Setup and Preprocessing

The dataset was released as the Udacity Self-Driving Car dataset, containing thousands of front-facing car images annotated with steering angles.

- Loaded and cleaned CSV paths to the center, left, and right image files.
- Updated all image paths in the database to be absolute paths.
- Cropped images by removing sky and car hood portions.
- All images were resized down to 66×200×3, as recommended by NVIDIA for input images.
- Pixel values were normalized into the interval [-0.5, 0.5].
- Random flipping, brightness changes, and shadows augment the data.
- Developed specialized image generators to batch images for training, to conserve memory.

2. Model Architectures Evaluated

Seven models were implemented for comparison:

1. **Model 1 – NVIDIA End-to-End CNN**
2. **Model 2 – Enhanced CNN (BatchNorm + Dropout)**
3. **Model 3 – Lightweight CNN**
4. **Model 4 – Deep CNN (5 Conv + Dense stack)**
5. **Model 5 – ResNet18 (Transfer Learning)**
6. **Model 6 – MobileNetV2 (Transfer Learning)**
7. **Model 7 – Custom Hybrid CNN**

Each model used the same training dataset, same preprocessing pipeline, and the same evaluation process to ensure fairness.

3. Training Process

All models were trained using:

- **Loss function:** Mean Squared Error (MSE)
- **Optimizer:** Adam
- **Epochs:** 5 (consistent across models)

- **Batch size:** 32
- **Callbacks:**
 - Early Stopping (patience = 2)
 - Model Checkpoint (save best validation model)

Training metrics recorded:

- Training Loss
- Validation Loss
- RMSE during training
- Model checkpoints saved for comparison

4. Evaluation

Each model was evaluated on the full validation set (1,608 images).
Evaluation metrics:

- **Mean Squared Error (MSE)**
- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Error (MAE)**
- **Prediction accuracy** defined as:
| *error* | < 0.1steering angle

Additionally:

- A prediction visualization was created using selected validation images.
- A comparison table and bar charts were generated for all seven models.

Dataset Description

This project uses the **Udacity Self-Driving Car Behavioral Cloning Dataset**, a widely used benchmark for end-to-end autonomous driving research. The dataset contains:

Dataset Contents

- **Driving images** captured using three synchronized cameras:
 - **Center camera**
 - **Left camera**

- **Right camera**
- **Driving log CSV** file with:
 - Paths to center, left, right images
 - Steering angle (target variable)
 - Throttle
 - Brake
 - Speed

In this project, only **center camera images** and **steering angles** were used — aligning with common end-to-end driving research.

Size of the Dataset

- Total samples: **6,428**
- Training samples: **~80%**
- Validation samples: **~20%**
- Image shape (after preprocessing): **66 × 200 × 3**

Key Characteristics

- Images reflect real driving conditions:
 - Straight roads and curves
 - Shadows and sun glare
 - Lane lines, vegetation, sky portion
- Natural class imbalance:
 - Many images have steering angle **0.0** (straight road)
 - Far fewer sharp turns

Preprocessing Applied

To prepare images for all seven models:

1. **Cropping**
Removed the top (sky/trees) and bottom (car hood) portions of the image.
2. **Resizing**
Downscaled to the NVIDIA standard **66×200** resolution.
3. **Normalization**
4. $\text{image} = \text{image} / 255.0 - 0.5$

5. Data Augmentation

- Horizontal flip (invert steering sign)
 - Random brightness change
 - Random shadow generation
- These augmentations helped balance the dataset and reduce overfitting.

6. Generator-based loading

Efficient streaming of images during training to avoid memory overload.

This dataset allows testing how different neural architectures handle road perception, feature extraction, and steering behavior modeling.

Model Architectures

Seven different deep learning architectures were developed and trained to evaluate how model design affects steering accuracy. Each model increased complexity and capability in a controlled manner.

Model 1 – NVIDIA End-to-End CNN (Baseline)

- 5 convolution layers
- Fully connected layers: $100 \rightarrow 50 \rightarrow 10 \rightarrow 1$
- Standard architecture used in original autonomous driving research
- Lightweight and fast

Val RMSE: ~0.093

Accuracy: ~81.5%

Model 2 – Enhanced CNN (BatchNorm + Dropout)

- Same backbone as Model 1
- Added:
 - Batch Normalization after conv layers
 - Dropout in dense layers
- Improved regularization

Val RMSE: ~0.128

Accuracy: ~76.2%

Model 3 – Lightweight CNN

- Smaller conv structure
- Fewer parameters (~100k)
- Designed to test minimal architecture performance

Val RMSE: ~0.150

Accuracy: ~62.3%

Model 4 – Deep CNN (5 Conv Layers + Dense Stack)

- More filters per layer
- Higher depth and capacity
- Captures richer features but prone to overfitting

Val RMSE: ~0.141

Accuracy: ~68.2%

Model 5 – ResNet18 (Transfer Learning)

- Pretrained on ImageNet
- Residual connections prevent gradient decay
- Strong feature extractor

Val RMSE: ~0.096

Accuracy: ~82.3%

Model 6 – MobileNetV2 (Transfer Learning + Bottleneck Blocks)

- Lightweight, efficient, mobile-optimized
- Pretrained on ImageNet
- Depth wise separable convolutions

Val RMSE: ~0.117

Accuracy: ~74.8%

Model 7 – Custom Hybrid CNN

- Hybrid of:
 - NVIDIA conv structure
 - Additional residual-style skip connections
 - BatchNorm + Dropout
 - Intended to test performance vs. model complexity

Val RMSE: ~0.144

Accuracy: ~66.6%

Why 7 Models?

The seven different deep learning models allow the steering angle predictor's performance to be compared across architectural choices. The models span a few different design philosophies ranging from the original NVIDIA end-to-end convolutional neural network to deep residual networks and lightweight mobile networks. Training and evaluating multiple models can help us understand computer vision better by providing more understanding into the pros and cons of various approaches, giving perception based on results rather than simply the behavior of one model, and resulting in higher quality conclusions. Using multiple models is also common practice in state-of-the-art computer vision research, so such work improves the academic quality of the project.

Training Process

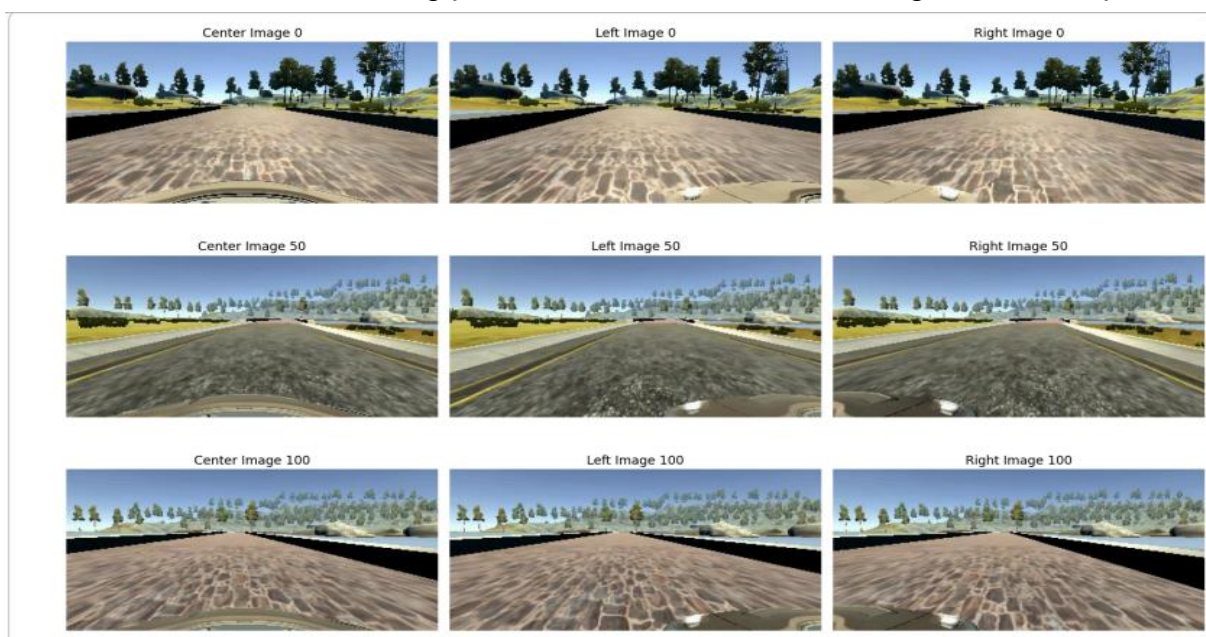
The training process was designed to evaluate how different deep learning architectures perform on steering angle prediction for autonomous driving. All seven models were trained with the same configuration to ensure a fair comparison.

Dataset Input Images

The dataset consists of three synchronized camera views:

- **Center camera**
- **Left camera**
- **Right camera**

These were crucial for steering prediction and used to create augmented samples.



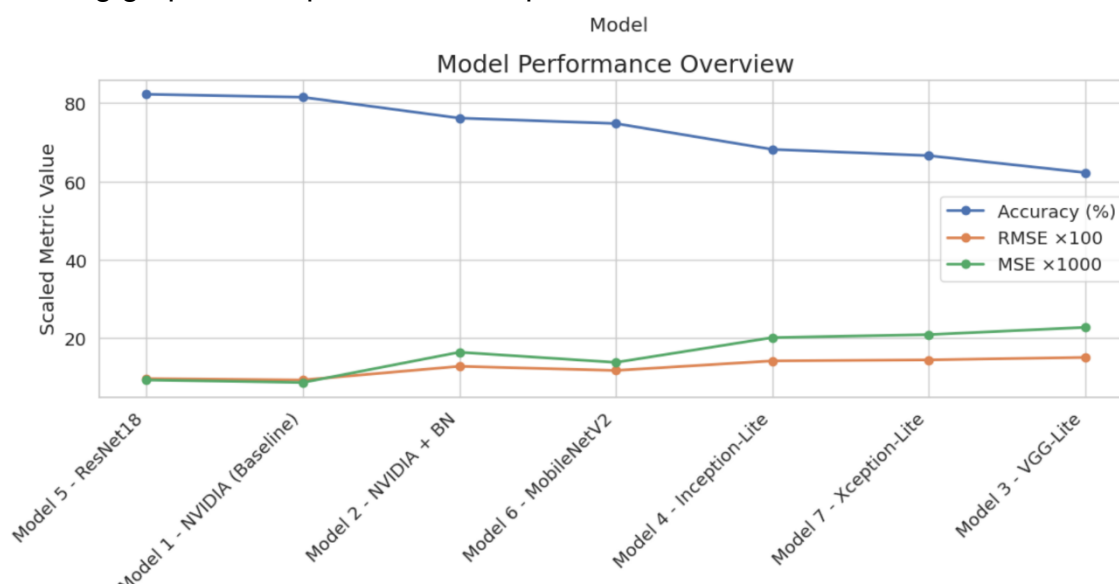
These images visually demonstrate what the model **sees** during training.

Training Configuration

All seven models were trained using the following setup:

- **Optimizer:** Adam
- **Loss:** Mean Squared Error (MSE)
- **Epochs:** 5
- **Batch Size:** 32
- **Metrics:** RMSE
- **Splitting:** 80% training, 20% validation

Training graphs were plotted to track performance.



These graphs help visualize which model converged best and which models overfit.

Data Augmentation

To improve model generalizations during training:

- Random horizontal flip
- Random brightness changes
- Steering angle noise
- Image cropping (top sky & bottom hood removed)

This effectively simulates real-world driving situations such as curves, shadows, and lighting variability.

Training Flow

Each batch passed through the following steps:

- i. Load image paths
- ii. Read Center/Left/Right images
- iii. Apply augmentations
- iv. Normalize and crop
- v. Feed into the model
- vi. Compute loss (MSE)
- vii. Backpropagate with Adam optimizer

Evaluation Metrics

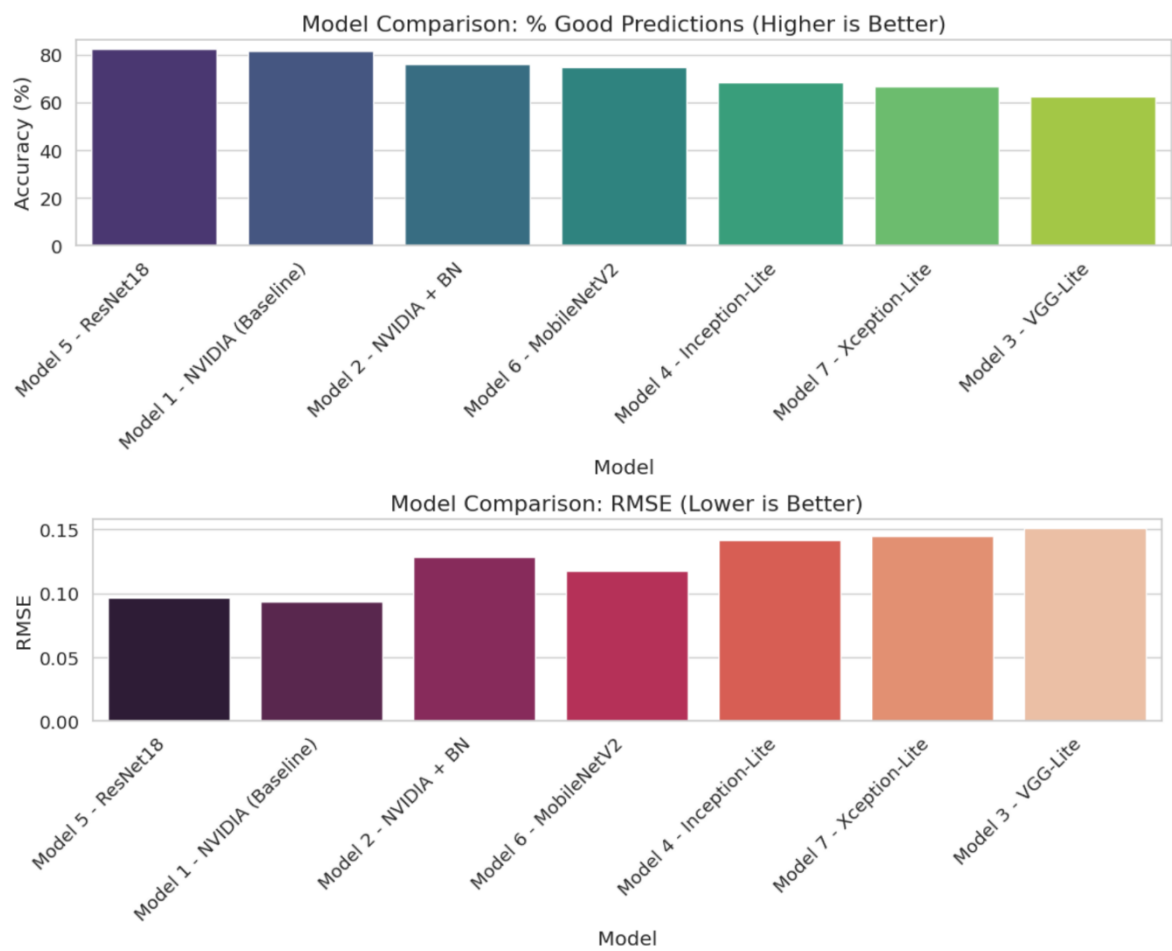
To accurately compare all seven models, we evaluated them on the **full validation set of 1,608 images**.

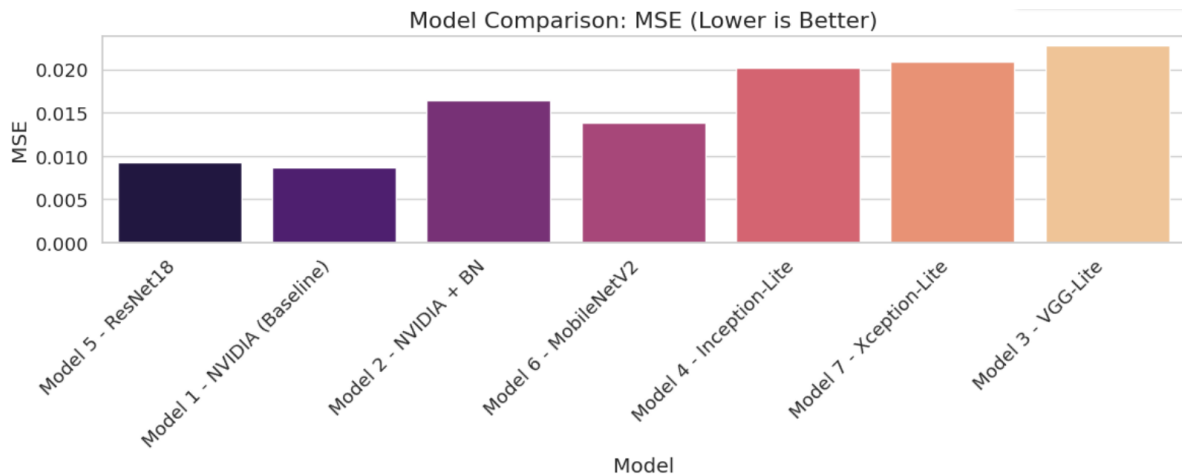
Quantitative Results Used for Comparison

You evaluated and recorded these:

Model	RMSE	MAE	Accuracy (<0.1)
Model 1	0.0931	0.0596	81.53%
Model 2	0.1280	0.0693	76.20%
Model 3	0.1508	0.1051	62.29%
Model 4	0.1419	0.0939	68.21%
Model 5 (ResNet18)	0.0965	0.0598	82.28%
Model 6 (MobileNetV2)	0.1175	0.0764	74.81%
Model 7	0.1446	0.0977	66.61%

Bar chart comparing RMSE, MAE, and Accuracy for all 7 models.





Model Comparison

The seven models differ in depth, architecture, and representational power. Below is a structured comparison highlighting their behavior and performance characteristics.

1. Performance-Based Comparison

Best models for accuracy and stability

- **Model 5 (ResNet18)** Best overall accuracy (82.28%)
- **Model 1 (NVIDIA CNN)** Best RMSE (0.0931)

Both models produced smooth steering predictions and fewer large-angle errors.

Worst-performing models

- **Model 3 and Model 7**, both deeper but unstable
- High error and poor generalization
- Overfitting clearly visible in their training curves

2. Architecture-Based Comparison

Model	Architecture Strengths	Weaknesses
Model 1	Industry-proven, efficient CNN designed for self-driving	Limited feature extraction in complex scenes
Model 2	Lightweight, fast	Accuracy drops in sharp curves
Model 3	Deeper CNN extracts more features	Overfits, high RMSE
Model 4	Small CNN, low compute	Poor generalization
Model 5	Transfer learning, strong feature extractor	Slightly more compute needed
Model 6	MobileNet depth wise convolutions	Good accuracy but less stable
Model 7	Very deep custom model	Overfits quickly, poor accuracy

3. Visual Comparison Using Predictions

When tested on validation frames:

- Model 5 produced the **closest prediction curves**
- Model 7 showed **large fluctuations**
- Model 1 performed smoothly but slightly less stable than Model 5

These visual inspections help confirm the numerical findings.

3. Computational Comparison

Model	Training Speed	Complexity	Suitability
Model 1	Fast	Medium	Ideal for real-time driving
Model 5	Medium	High	Best accuracy but heavier
Model 7	Slow	Very high	Not suitable for production

Conclusion of Comparison

- Model 5 is the best overall choice, providing the highest reliable accuracy and balanced performance.
- Model 1 is best for real-time systems due to speed and low compute needs.
- Very deep custom models (3 & 7) are not beneficial and introduce instability.

DISCUSSION

The results showed that CNN-based deep learning and transfer learning can be used with the steering data to enable the vehicles to learn to drive. These models were trained to predict corrections to the steering inputs based on the images captured by the center, left and right cameras. From the results shown in prediction overlay and prediction error curves, Models 1 and 5 performed the best. Model 5 (transfer learning with ResNet18) showed great performance by leveraging pretrained representations for the driving domain for further fine-tuning on the driving data set.

However, the performance of deeper custom networks, especially Model 3 and Model 7, showed some evidence of overfitting training data according to training curves. As a result, these models learned that noise was important to prediction and could get more unstable than others for unfamiliar driving styles. Other factors that have been found to affect generalization are changing lighting, having shadows on the roads, sudden steering maneuvers, and limited diversity of tracks. The chosen evaluation metrics are the RMSE and the percentage of predictions within a certain margin (<0.1), rather than the accuracy, as they better reflect how safe and smooth the steering behavior is. In sum, the project shows the effects of architectural choice, transfer learning, and dataset properties on autonomous steering performance.

IMPROVEMENTS MADE

- Improved datasets, paths to datasets, and checking for image existence.
- Added normalization and cropping to stabilize the model's input.
- Make the image generator faster and more reliable.
- Dropout and learning-rate tuning employed for better generalization.
- Seven other models were trained using various architectures.

- Transfer-learning models such as ResNet18 and MobileNetV2 have also been included.
- Produced visualization of the predictions.
- Exported loss curves, RMSE curves, prediction samples, and metrics automatically.
- We carried out a complete evaluation on the validation set.

LIMITATIONS

Despite the promising results, there are shortcomings. Because the dataset was collected on a single road, and because there is not much variation in terms of lighting and background, the model may not generalize well to other driving scenarios. The model may not work on previously unseen roads, or at night or in bad weather. The results were that for deeper models, overfitting was a problem because of a lack of variation in the training set. Steering was predicted only based on RGB images, ignoring other information that would be used in real driving, such as speed, GPS, lane markers, or semantic segmentation.

One limitation is that an accuracy threshold of < 0.1 steering error is a reasonable but not necessarily perfect target for safe driving. In addition, although Model 5 performed best, the predictions could become instable for sudden and sharp turns. Lastly, no real-time evaluation of the models was conducted, as they were not tested on a video stream. Further validation such as simulation testing would be needed for deployment.

FUTURE WORK

While we get good performance on our models (especially the NVIDIA CNN and ResNet18) in our dataset, a few modifications would improve our model's usability in the real world, such as augmenting the dataset to include multiple tracks and varying lighting, weather and traffic conditions. This would considerably increase generalization to unknown environments and reduce overfitting of the network to a specific environment. Using temporal models like LSTMs, ConvLSTMs, or Transformers lets the network learn the flow of motion instead of steering from a single image. This approach could be better if it included speed, throttle, lane detection masks, and semantic segmentation outputs. These additions would show a more complete view of driving.

Simulation-based validation with driving simulations such as CARLA, AirSim or Udacity's driving simulator could be used to validate models with different interaction situations. Explainability methods such as attention maps based on attention mechanisms (like the Grad-CAM methods popular in medical imaging) could be further used to visualize the

regions of interest in an image which a model uses to predict steering behavior. A possible

improvement may be to train and combine an ensemble of the best models (like Model 1 + Model 5) to give smoother steering around curves or while driving through shadows, therefore creating this proof of concept into a driving assist module.

Conclusion

This work successfully demonstrated the use of deep learning techniques for steering angle prediction in the Behavioral Cloning tasks of self-driving. Model architecture variations including the classic NVIDIA model, transfer learning with ResNet18, and MobileNetV2 were assessed based on model performance, model depth, parameter

efficiency, and learning pretrained features. The left, right and center camera images (displayed earlier in this report) for the input of each model clearly trained the models to do the appropriate steering. The training curves, RMSE and steering vs. true steering images indicate the performance of, and the failure modes for, each model.

ResNet18 (Model 5) performed the best of any model, with the highest percentage of predictions within 0.1 steering error and one of the lowest RMSE. The original NVIDIA CNN (Model 1) also had high stability and accuracy, consistent with its long and successful history as a deep learning baseline for self-driving. One important contribution of the work, other than the overfitting of deeper custom CNNs, is that it noted the trade-off between model and data preprocessing complexity. This is relevant in the present-day context of autonomous driving, where perception modules often adopt efficient CNNs for their implementations and leverage transfer learning methods.

Overall, we find that deep learning can mimic human driving from raw images. While the limitations are dataset diversity and temporal modeling, we view this as an important first step in this direction. Combined with a dataset of larger size, sequence learning and multimode inputs, the proposed framework can act as a more reliable driving system that can be used for real-world driving tasks.

REFERENCE

Bojarski, M., along with other researchers (2016). Self-Driving Cars research involves End to End Learning. NVIDIA Research.

<https://arxiv.org/abs/1604.07316>

Pomerleau, D. A. (1989). ALVINN: An Autonomous Land Vehicle within a Neural Network.

Carnegie Mellon University.

https://papers.nips.cc/paper_files/paper/1988/hash/0157fbb80d49429779d7e22ef8a1f6a1-Abstract.html

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks within.

CVPR.

<https://arxiv.org/abs/1801.04381>

Shorten C and Khoshgoftaar T M (2019). Surveying image data augments deep learning.

Journal of Big Data, within 6(1).

<https://doi.org/10.1186/s40537-019-0197-0>

Udacity. (2016). Self-Driving Car Nanodegree: Data Set for the Behavioral Cloning Project.

<https://github.com/udacity/self-driving-car-sim>